Kartik Shrivastava
Graduate: B. Tech Computer Science (Hons.) LPU, India (2016-2020) Grade 9.14/10 **Past work(full time):** Computer Graphics & Research Engineer @ Viga, Bangalore, India
Current work(part time): Working on a personal startup idea Dietitian Near Me
Email:
Github: github.com/KartikShrivastava
Linkedin: linkedin.com/in/kartik-techiegeekyfreak/
Discord Nick: KartikShrivastava (ylang ylang#3073)

Add Tiled compatibility to/from Enigma

10th April 2022

Abstract

Enigma is an ever expanding open source game development environment which comprises its new IDE RadialGM (old IDE is LateralGM) + Compiler + Game Engine. Enigma is widely used as an open source alternative to older versions of Game Maker.

Tiled is an open source level editor which has become an increasingly popular level editor in the game development and level editing community.

Enigma's future plan includes becoming more independent by shifting focus away from Game Maker. As Tiled offers better room editing features than Game Maker, adding Tiled compatibility to Enigma can help users take advantage of Tiled room editor along with Enigma's development environment. This project also expects some research to know the intermediate representation of Tiled in much more detail.

TMX is a Tiled specific room format and **EGM** is an Enigma specific project format. Most of the features supported by EGM format are based on Game Maker's format **GMX**. The way this project aims to achieve Tiled compatibility is twofold. First part consists of adding a TMX importer to Enigma while the second part consists of adding an EGM importer to Tiled. Followed by adding Gtests in libEGM to check deserializing took place appropriately.

Milestones overview

1. **Before coding period:** Determine the structure of Tiled's data layer(TMX), its tileset extension TSX and note differences between mentioned Tiled formats and EGM.
2. **Enigma specific:** Update room protocol buffer of Enigma to handle the differences between room formats of Tiled and Enigma.
3. **Enigma specific:** Complete protocol buffer based deserializer to import Tiled room format TMX into Enigma.
4. **Tiled specific:** Integrate protocol buffers to Tiled fork.
5. **Tiled specific:** Implement a protocol based deserializer to import Enigma room format EGM into Tiled fork.
6. **Testing:** Test the protocol buffers loaded from EGM and TMX, field by field comparison.
7. **Stretch goal:** Add support to missing Tiled features in Enigma's room protocol buffer.

Timeline and Milestones breakdown

Application period - 20 May

1. Take the help of mentors to understand implementation steps in detail and further break down milestones into simpler but detailed steps.
2. Refer to GMX 1.4 documentation to understand usage of several unused fields in existing protos. Also refer to Tiled GMX plugin implementation to understand how Tiled currently maps its Elements and fields to export a GMX file.
3. Find missing analogies between TMX and EGM with the help of mentors.
4. Write a prototype for a sample TMX/TSX file to fill a single EGM Resource and get an idea of deserializer implementation.
5. Fix existing protocol buffer related and other relevant bugs in enigma-dev or RadialGM projects to understand Enigma a little bit more.

20 May - 12 June (Community bonding period)

1. Understand working of grpc.
2. Understand working of protocol buffers in detail.
3. Improve the prototype implementations to discover any hidden complexity.
4. Discuss with mentors to find and fix roadblocks ahead of the coding period.
5. Discuss and document integration of protocol buffers in Tiled with mentors.
6. Discuss and document EGM reader implementation in Tiled with mentors.

13 June - 4 July (Coding period starts: Enigma specific milestone 1)

Update the existing protocol buffers by reusing the existing proto fields to carefully match with Tiled fields.

- This will be the most involved milestone as TMX/TSX and EGM have a lot of interdependent elements/resources which need to be matched appropriately(if possible) otherwise new proto fields should be added. These updated protos would eventually merge to the master branch once testing and validation is done. Analogy table (mentioned in the Miscellaneous section) would be used for this task.

5 July - 19 July (Enigma specific milestone 2)

Implement a TMX reader and check the compatibility of updated protocol buffers by manually comparing the values of existing EGM buffer and buffer loaded from TMX.

1. This step includes adding a new source file for the TMX/TSX reader in libEGM. I'll take a reference of tinytmx c++ library for parsing map generated by TMX.
2. Please note that all newly added proto fields (additional features of Tiled) do not yet have an actual representation in the Enigma IDE or engine itself, it will be covered as a stretch goal of this project.

20 July - 24 July

Buffer for backlog work of previous milestones.

25 July - 29 July (Phase-I evaluation period)

25 July - 15 August (Tiled specific milestone 1)

1. Fork the Tiled project and proceed with the changes in a separate branch.
2. Add support to the protocol buffer in the Tiled fork branch.
3. Create EGM file deserializer by adding a new EGM "plugin" in Tiled. (Tiled prefers to adds import/export functionality via plugins)
    a. Add libEGM dependency to this plugin
    b. Use egm.h from libEGM to deserialize EGM files into a protocol buffer.
4. Now fill the Tiled intermediate map format using a newly created proto project object. Check the compatibility of protocol buffers by manually comparing the values of existing TMX and loaded EGM buffers.

16 August - 23 August (Tiled specific milestone 2 and Testing)

Add Gtests to Enigma to automate the buffer similarity checks.

- With the help of mentors I'll write a Gtest to check whether EGM and TMX buffers are similar or not. It will be done by loading both buffers at the same time and testing the similarity by iterating through all the fields. Reference: https://github.com/enigma-dev/enigma-dev/blob/master/CommandLine/emake-tests/event-reader-tests.cpp

24 August - 4 September

Buffer for backlog work of previous milestones and beginning of new Tiled feature addition to Enigma/RadialGM.

5 September - 19 September (Phase-II evaluations)

20 September - TBD (Stretch goals)

1. Discuss with mentors what are the most important missing Tiled features to be integrated into Enigma.
    a. As level editors of both Tiled and Enigma currently have feature parity where even after successfully loading a TMX project into Enigma all features of a given TMX project could not be one-to-one translated.
2. Implement the missing features and required GUI for the same.

Prototype code for loading Tiled TSX

Following piece of code demonstrates an example of how TSX format of Tiled (which stores Tileset used in maps separately) can be deserialized and stored in Background.proto. This prototype uses an extremely simple version of Tileset. And there is no modifications made in Background.proto as for this simple TSX file most nodes and attributes were able to establish relation with existing proto.

Content of **TSX File** used:

```
<?xml version="1.0" encoding="UTF-8"?>
<tileset version="1.8" tiledversion="1.8.4" name="trashset" tilewidth="32"
tileheight="32" spacing="1" margin="1" tilecount="48" columns="8">
 <image source="../examples/tmw_desert_spacing.png" width="265" height="199"/>
</tileset>
```

tmx.h header file:

```
#ifndef EGM_TMX_H
#define EGM_TMX_H

#include "file-format.h"

namespace egm {
class TMXFileFormat : public FileFormat {
public:
  TMXFileFormat(const EventData* event_data) : FileFormat(event_data) {}
  virtual std::unique_ptr<Project> LoadProject(const fs::path& fPath) const
override;
};
} // namespace egm
#endif
```

tmx.cpp source file:

```
#include  "tmx.h"
#include "action.h"
#include "strings_util.h"
```

```
#include "pugixml.hpp"

//..STL HEADERS..OMITTED FOR SPACE

using namespace buffers::resources;

namespace egm{
namespace {

class tsx_tileset_loader : public pugi::xml_tree_walker {
public:
  tsx_tileset_loader(buffers::resources::Background *bg, const fs::path& fPath) {
    background = bg;
    background->set_use_as_tileset(true);
    parentDirPath = fPath.parent_path().u8string()+"/";
  }

  buffers::resources::Background *background;
  std::string parentDirPath; // useful in loading image correctly

private:
  virtual bool for_each(pugi::xml_node& node) {
    for(pugi::xml_attribute attr : node.attributes()) {
      if(strcmp(node.name(),"tileset") == 0){
        if(strcmp(attr.name(),"tilewidth") == 0)
          background->set_tile_width(std::stoi(attr.value()));
        else if(strcmp(attr.name(),"tileheight") == 0)
          background->set_tile_height(std::stoi(attr.value()));
        else if(strcmp(attr.name(),"spacing") == 0){
          background->set_horizontal_spacing(std::stoi(attr.value()));
          background->set_vertical_spacing(std::stoi(attr.value()));
        }
        else if(strcmp(attr.name(),"margin") == 0){
          background->set_horizontal_offset(std::stoi(attr.value()));
          background->set_vertical_offset(std::stoi(attr.value()));
        }
      }
      else if(strcmp(node.name(),"image") == 0){
        if(strcmp(attr.name(),"source") == 0){
          std::string absoluteImagePath = parentDirPath+attr.value();
          background->set_image(absoluteImagePath);
        }
        else if(strcmp(attr.name(),"width") == 0)
          background->set_width(std::stoi(attr.value()));
        else if(strcmp(attr.name(),"height") == 0)
          background->set_height(std::stoi(attr.value()));
      }
    }
```

```
    return true;
  }
};
} // anonymous namespace

std::unique_ptr<buffers::Project> TMXFileFormat::LoadProject(const fs::path& fPath)
const {

  pugi::xml_document doc;
  if(!doc.load_file(fPath.u8string().c_str())) return nullptr;

  auto proj = std::make_unique<buffers::Project>();
  buffers::Game *game = proj->mutable_game();
  auto root = game->mutable_root();
  root->set_name("tsxRootTest");

  auto background = root->mutable_folder()->add_children();
  background->set_name("tsxBackgroundTest");

  tsx_tileset_loader backgroud_walker(background->mutable_background(), fPath);
  doc.traverse(backgroud_walker);

  return proj;
}

} // namespace egm
```

Contributions to enigma-dev and RadialGM

1. Enigma-dev: BMP write bug fix #2101 (Merged): Updated existing BMP header and save function to support transparency.
2. Enigma-dev: Fix 'folder replaces resource' egm-read issue #2293 (Open): A small fix for the issue in which on loading egm projects in RGM, folders appear in place of actual resources.
3. RadialGM: Fix for duplicate resource name issue #223 (Open): Fix for a bug where new resources of the same type were being created with the same name. This fix enforces new resource names that are unique.
4. Enigma-dev/RadialGM: Work In Progress: Need proper defaults in protos #1872

Miscellaneous

TMX and EGM Analogy

Following table mentions analogies between Tiled TMX and Enigma EGM format, this helps in understanding the differences between both the formats.

Note: This is an attempt to find as many analogies before the proposal period ends.

Special symbols used and their meaning in “EGM Resource analogous to TMX Element” column:

1. -> : Follow the resource hierarchy path till all -> get exhausted.
2. [ ] : Array of the resource fields.
3. + : Additional setting on right for the Resource present on the left.
4. - : Analogy not found yet
5. x : Analogy does not exist
6. + : Possibility of a new proto

TMX (Tiled 1.8)				EGM 2
Sr No.	**TMX Element**	**0 or 1 contained element**	**0 or more contained elements**	**EGM Resource analogous to TMX Element**
1.	map	properties	tileset	Room
		editorsettings	layer	
			objectgroup	
			imagelayer	
			group	
2.	editorsettings	chunksize		-
		export		
3.	tileset	image	tile	Background + Use as Tileset = true
		tileoffset		
		grid		
		properties		
		terraintypes		
		wangsets		
		transformations		

4.	tileoffset			Background –> [Horizontal offset, Vertical offset]
5.	grid			-
6.	image	data		
7.	terraintypes		terrain	-
8.	terrain	properties		-
9.	transformations			-
10.	tile(contained within tileset only)	properties		+
		image		
		objectgroup		
		animation		
11.	animation		frame	Timelines
12.	frame			
13.	wangsets		wangset	x
14.	wangset	properties	wangtile	x
	(Extra: can contain upto 255 wangcolor)			
15.	wangcolor	properties		x
16.	wangtile			x
17.	layer	properties		Room –> Tiles –> Current Tile Layer
		data		
18.	data		tile	-
			chunk	
19.	chunk		tile	-
20.	tile(contained within data and chunk)			Room –> Tiles
21.	objectgroup	properties	object	-
22.	object	properties		-
		ellipse		
		point		
		polygon		
		polyline		

		text		
23.	ellipse			-
24.	point			Path
25.	polygon			Path
26.	polyline			Path
27.	text			-
28.	imagelayer	properties		-
		image		
29.	group	properties	layer	-
			objectgroup	
			imagelayer	
			group	
30.	properties		property	-
31.	property	properties		-

A little about me

-

Miscellaneous-2

- **My motive to be a GSoC student:** I always wanted to become a better open source contributor in the game engine domain. And seeing a welcoming environment of Enigma GSoC to non-student developers like me who want to receive mentorship and at the same time want to connect and contribute to open source community in the field of their wish is really an opportunity I do not want to miss.
- **My commitments towards the GSoC project:** I'll be able to dedicate 30-40 hours per week for this GSoC project. And just to mention the only other part-time commitment I have is to give a push to a startup idea for which I can adjust timings without much trouble.